Token-Cost-Aware W-TinyLFU Eviction Policy for GPTCache

Amit Abramovich, Noa Yosipov Berenshtein

Contents

1 Introduction

1.1 Problem Statement

Large Language Model (LLM) applications increasingly rely on semantic caching to reduce latency and API costs. GPTCache, an open-source semantic cache by Zilliz, intercepts LLM API calls, encodes queries as embeddings, and serves cached responses for semantically similar queries. However, GPTCache's built-in eviction policies (LRU, FIFO, LFU, Random Replacement) treat all cached entries as equal, ignoring a critical property of LLM workloads: **response regeneration cost varies enormously**. A 2000-token response is far more expensive to recompute than a 20-token one. When the cache is full and an entry must be evicted, a cost-unaware policy may discard an expensive response while retaining a cheap one—wasting the regeneration cost that was already paid. This project implements a **Token-Cost-Aware W-TinyLFU** eviction policy that combines frequency-based admission filtering with cost-weighted eviction decisions, preferentially retaining high-cost cached responses.

1.2 Related Work

The Window TinyLFU (W-TinyLFU) algorithm was introduced by Einziger, Friedman, and Manes [1] and serves as the eviction policy in Caffeine, Java's most widely used caching library. W-TinyLFU combines a small window LRU with a TinyLFU admission filter and a segmented LRU main cache, achieving near-optimal hit rates across diverse workloads. The GPT Semantic Cache paper [2] studied similarity thresholds for LLM caching and identified 0.80 as the optimal cosine similarity threshold, achieving up to 68.8% cache hit rate with 97%+ accuracy. Our contribution extends W-TinyLFU with LLM-specific cost awareness—to our knowledge, the first integration of cost-weighted W-TinyLFU into an LLM semantic caching system.

1.3 Baseline Framework Selection

We selected **GPTCache** (v0.1.44, MIT License) as our baseline. Table 1 summarizes its main features.

Table 1: GPTCache main features.

Feature	**Detail**
Caching Model	Semantic (embedding-based) + exact match
Embedding Support	OpenAI, ONNX, Sentence-Transformers
Scalar Storage	SQLite (default), MySQL, PostgreSQL
Vector Storage	FAISS (default), Milvus, Chromadb
Default Eviction	**LRU** via `cachetools.LRUCache`
Also Supports	FIFO, LFU, Random Replacement
Language / Stars	Python (100%), ∼8 000 GitHub stars, MIT

Why GPTCache:

1. **Clean eviction abstraction** — the `EvictionBase` class requires only `put`, `get`, and `policy` methods, isolated in `gptcache/manager/eviction/`.
2. **Explicit gap in eviction sophistication** — current policies ignore access frequency, entry cost, and entry size; the roadmap lists cost-aware eviction as a development goal.
3. **Built-in semantic matching pipeline** — embedding generation, vector search, and similarity evaluation are already implemented.
4. **Mature and stable codebase** — comprehensive test suite (`pytest`), documentation, and stable since August 2024.

Alternatives considered:

Library	Why Not Selected
LangChain Cache	Eviction delegated to backend (Redis, etc.), not a first-class concept
vLLM / SGLang	KV cache at inference level; no pluggable eviction policy
LMCache	Tied to GPU inference engines (vLLM/SGLang)
ModelCache	Smaller community, less clean eviction abstraction

2 Extension Design

2.1 Motivation

Standard eviction policies (LRU, LFU, FIFO) treat all cache entries as equally valuable. In LLM caching, this assumption is incorrect: a cached response with 2 000 tokens costs ∼100× more to regenerate than one with 20 tokens. Our extension makes the cache "cost-aware" by incorporating the response token count into eviction decisions.

2.2 Architecture

Our W-TinyLFU implementation follows the paper's three-segment architecture, illustrated in Figure 1.

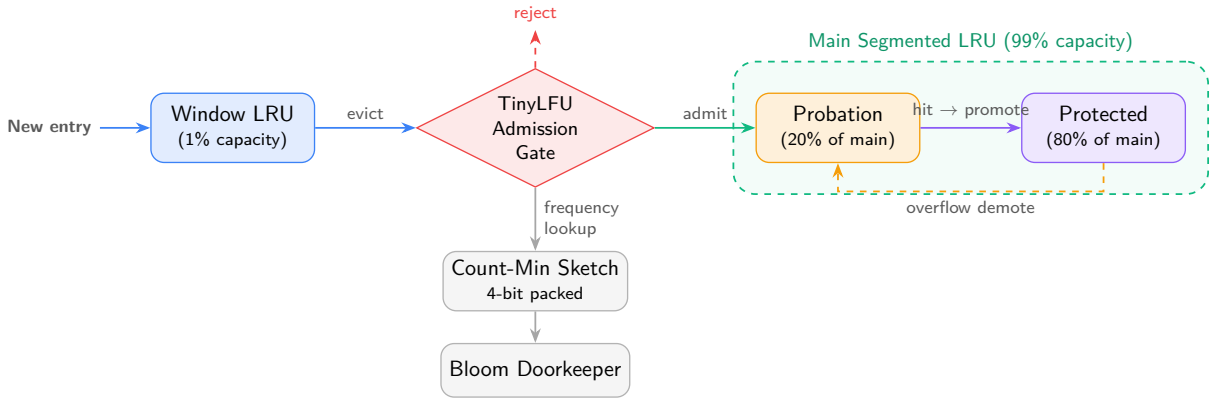


Figure 1: W-TinyLFU three-segment architecture. New entries enter the window cache. When the window overflows, its LRU victim becomes a candidate for the main cache. If the main cache is full, the candidate competes against the probation segment's LRU victim via the TinyLFU admission filter.

2.3 Supporting Data Structures

1. **Count-Min Sketch (CMS)**: 4-bit packed counters (16 per uint64 word) with 4 hash functions. Maximum counter value 15. Periodic aging halves all counters via `(table » 1) & 0x7777777777777777` when additions reach 10× cache capacity.

2. **Bloom Filter Doorkeeper**: Filters one-hit-wonders. Cleared whenever the CMS resets, per the TinyLFU paper.

3. **Segmented LRU (SLRU)**: Probation (20%) + protected (80%). Items enter probation; on hit, promoted to protected. Protected overflow demotes LRU victim to probation.

2.4 Cost-Aware Admission

When `cost_aware=True`, the admission decision computes:

$$\text{value} = \text{frequency} \times \text{cost}$$

where cost is the response token count. Following Caffeine's `BoundedLocalCache.admit()`: if $\text{value}_{\text{candidate}} > \text{value}_{\text{victim}}$, admit; if frequency ≥ 6, admit with $\sim 1/128$ probability (hash-DoS defence); otherwise reject.

2.5 API Compatibility

Our policy maintains full compatibility with GPTCache's `EvictionBase` interface, registered as `name="wtinylfu"` with the same `put`/`get`/`policy` API. The `set_cost()` method is an additive extension.

2.6 Tunable Parameters

Table 2: W-TinyLFU tunable parameters.

Parameter	Default	Description
`window_pct`	1.0%	Window cache as % of total capacity
`probation_pct`	20.0%	Probation segment as % of main cache
`cost_aware`	True	Enable cost-weighted eviction
`cms_width_multiplier`	1	CMS width scaling factor
`reset_multiplier`	10	CMS aging interval

3 Experimental Setup

3.1 Workload Profiles

1. **Synthetic (Zipfian)**: 3 000 queries from a Zipfian distribution ($\alpha = 0.7$, vocabulary = 500). Response tokens: 17–393 (mean 174). Stresses hit/miss ratio via skewed access patterns.

2. **LMSYS-Chat-1M** [3]: 3 000 real LLM conversations. High diversity, most queries unique. Measures cache overhead under realistic conditions.

3. **Ablation profiles**: "repetitive_short" (vocab = 100, $\alpha = 1.5$, 10 000 samples) and "novel_long" (vocab = 5 000, $\alpha = 0.5$, 10 000 samples).

3.2 Metrics Collected

Cache hit rate, token saving ratio, per-request latency (p50/p95/p99/mean), throughput (QPS), peak memory (MB), CPU utilisation (user + system seconds via `os.times()`).

3.3 Automation and CI

Benchmarks are automated via Python scripts supporting parallel execution (`-workers -1`) and repeated trials (`-repeats 3`) for statistical significance. GitHub Actions CI pipeline runs tests and benchmarks. Docker container runs the full evaluation in one command.

3.4 Configuration

- **Policies**: LRU, FIFO, LFU, W-TinyLFU, W-TinyLFU + Cost
- **Cache sizes**: 50, 100, 200 entries
- **Thresholds**: 0.80 (SOTA [2]) and 0.85
- **Embedding**: all-MiniLM-L6-v2 (384-dim), FAISS flat, SQLite in-memory
- **Repeats**: 3 trials with shuffled query order per trial
- **Warmup**: 2× cache size queries excluded

4 Results

All results are reported as mean ± std over 3 independent trials (shuffled query order per trial).

4.1 Synthetic Workload (threshold = 0.85)

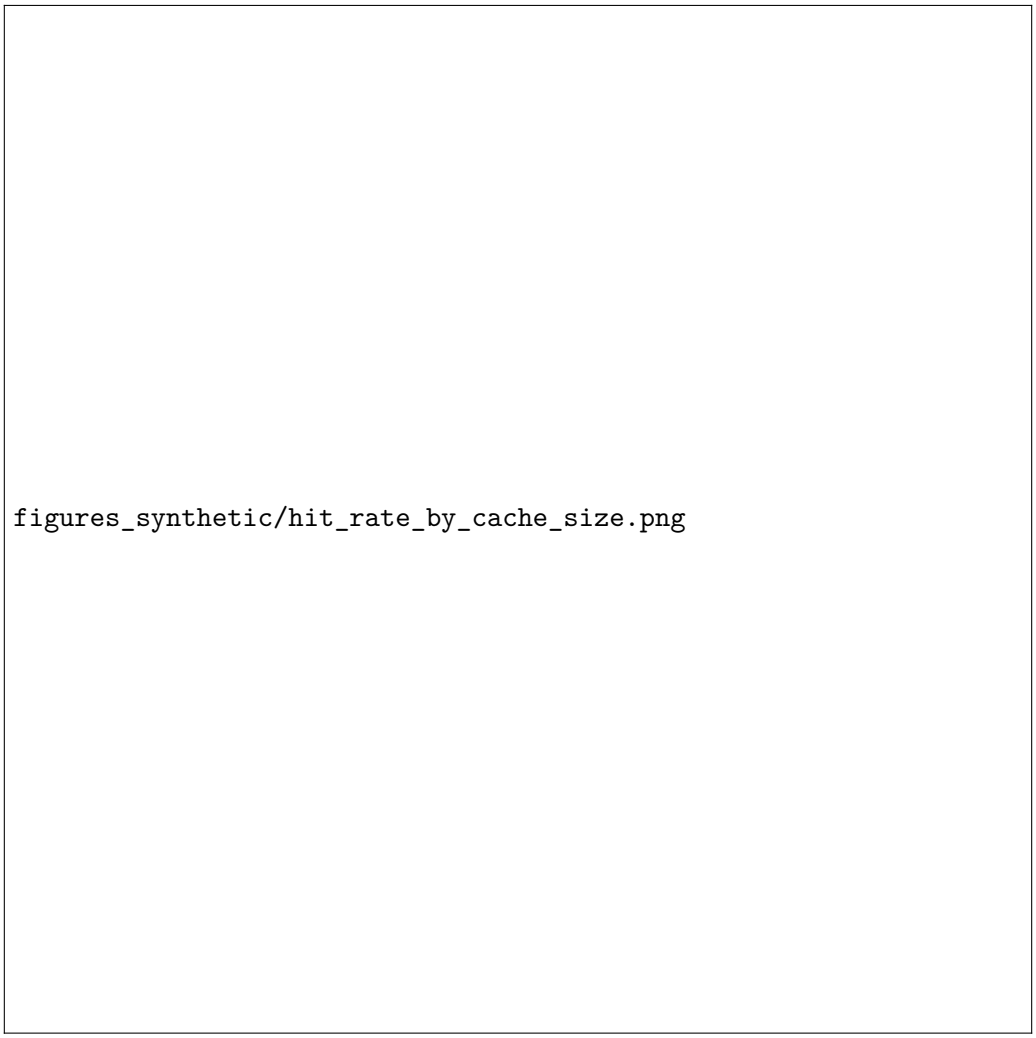


Figure 2: Hit rate by cache size on synthetic Zipfian workload (threshold = 0.85).

Figure 3: Token saving ratio by cache size (synthetic). The gap between hit rate and token savings confirms cost-aware caching of expensive responses.

Table 3: Synthetic workload results (mean ± std, 3 trials, threshold = 0.85).

	cs = 50		**cs = 100**		**cs = 200**	
Policy	Hit Rate	TokSave	Hit Rate	TokSave	Hit Rate	TokSave
LRU	30.5 ±0.5	44.0 ±0.2	49.6 ±1.8	67.3 ±2.2	82.5 ±6.8	93.1 ±3.6
FIFO	27.6 ±0.4	38.6 ±0.7	44.3 ±0.5	58.0 ±0.6	69.9 ±0.6	80.9 ±0.3
LFU	50.5 ±1.1	71.7 ±2.1	59.5 ±5.7	79.8 ±4.0	83.5 ±5.7	94.7 ±2.1
W-TinyLFU	50.4 ±0.5	70.3 ±0.3	68.5 ±0.4	85.4 ±0.7	93.5 ±0.7	98.3 ±0.2
W-TinyLFU+Cost	**53.8** ±2.5	**76.4** ±3.5	**71.3** ±0.8	**88.9** ±0.8	**91.9** ±4.2	**97.7** ±1.5

At cs = 50, W-TinyLFU + Cost saves 76.4 ±3.5% of tokens vs. 44.0 ±0.2% for LRU—a **74% relative improvement** with non-overlapping confidence ranges.

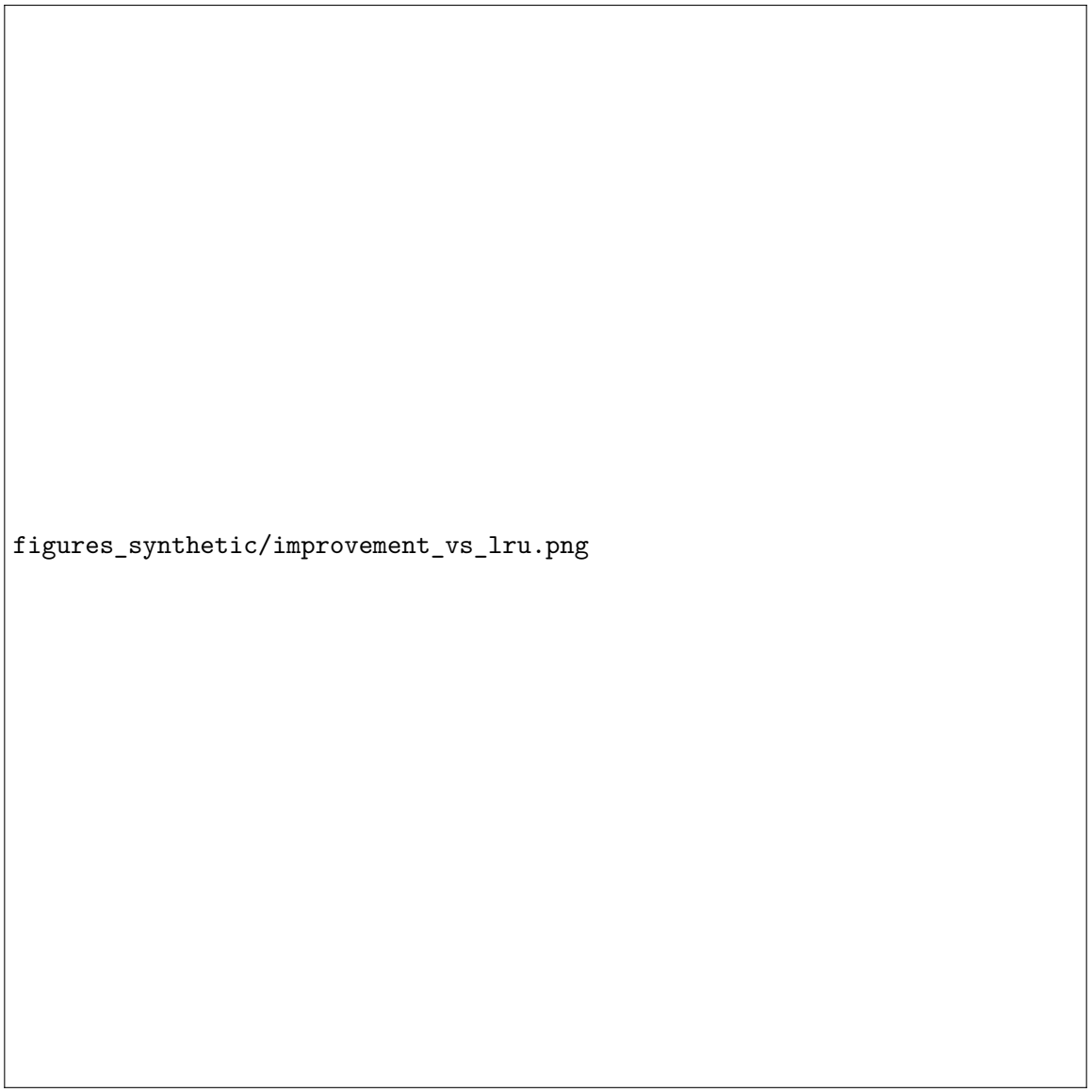


Figure 4: Percentage improvement over LRU baseline (synthetic).

4.2 Latency and Resource Analysis (Synthetic)

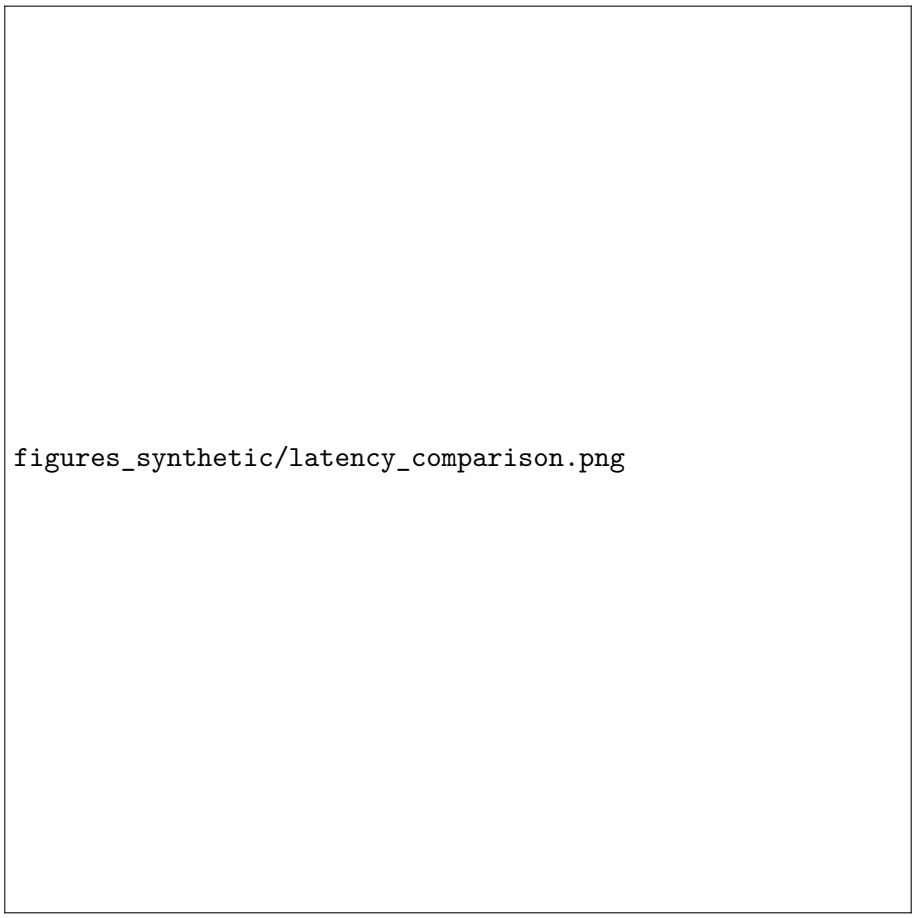


Figure 5: Per-request latency percentiles at cache size 200 (synthetic).

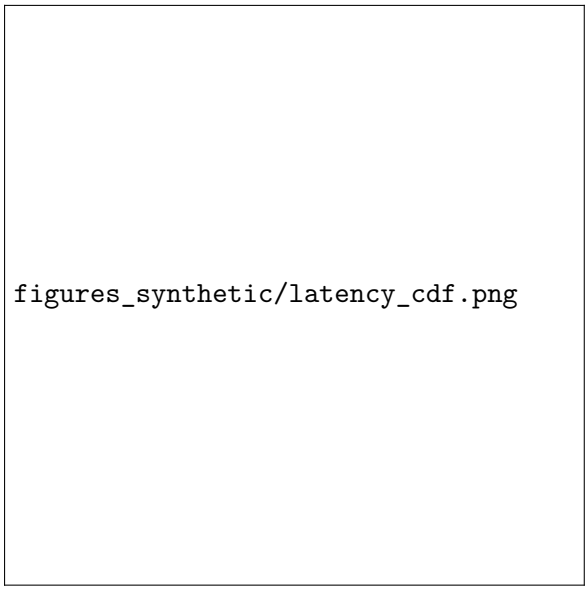
Table 4: P95 latency change vs. LRU (synthetic).

	cs = 50		**cs = 100**		**cs = 200**	
Policy	p95 (ms)	vs LRU	p95 (ms)	vs LRU	p95 (ms)	vs LRU
LRU	111.9	–	82.5	–	74.0	–
FIFO	117.0	+4.6%	86.3	+4.6%	71.9	−2.8%
LFU	96.4	−13.9%	80.7	−2.2%	70.3	−5.0%
W-TinyLFU	159.6	+42.6%	109.0	+32.1%	52.7	−28.8%
W-TinyLFU+Cost	154.0	+37.6%	113.9	+38.1%	65.8	−11.1%

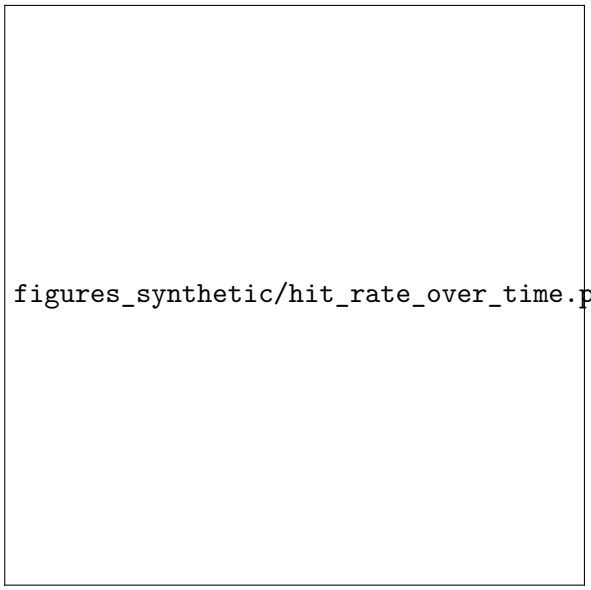
W-TinyLFU adds 38–43% overhead at p95 for small caches (cs = 50). At cs = 200, the overhead disappears—W-TinyLFU is actually *faster* because its higher hit rate reduces cache insertions. Even the worst-case overhead (∼40 ms) is negligible vs. an LLM API call (100–2 000 ms).

Table 5: CPU utilisation (total seconds, synthetic).

Policy	**cs = 50**	**cs = 100**	**cs = 200**
LRU	138.2	151.8	156.1
FIFO	137.2	156.4	152.2
LFU	145.2	160.2	153.8
W-TinyLFU	194.2 (+41%)	172.8 (+14%)	166.2 (+6%)
W-TinyLFU+Cost	193.9 (+40%)	181.1 (+19%)	158.8 (+2%)



(a) Latency CDF (cache size = 200).



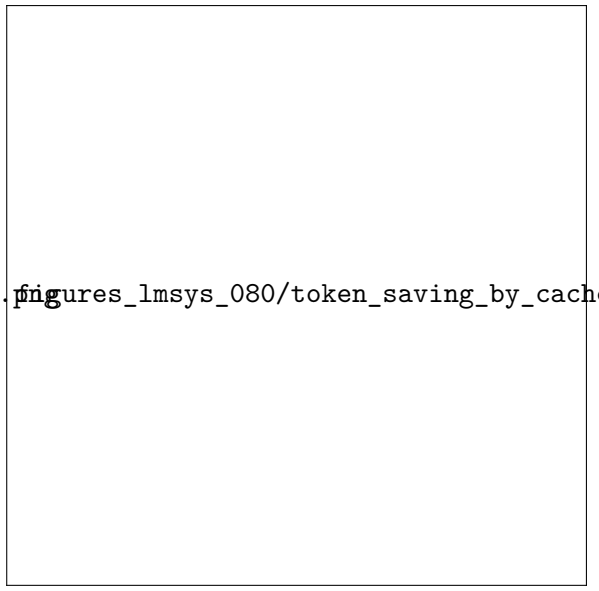
(b) Hit rate over time (sliding window = 200).

Figure 6: Latency distribution and temporal hit rate behaviour (synthetic, cs = 200).

4.3 LMSYS-Chat-1M Real Data (threshold = 0.80)

This experiment answers: *Does the improvement hold on real, diverse LLM conversations?* We use threshold 0.80, identified as the SOTA optimal by [2].

(a) Hit rate by cache size.



(b) Token saving ratio by cache size.

Figure 7: LMSYS-Chat-1M results (threshold = 0.80). LFU achieves highest hit rate; W-TinyLFU+Cost achieves highest token savings at small cache sizes.

Table 6: LMSYS results (mean ± std, 3 trials, threshold = 0.80).

	cs = 50		**cs = 100**		**cs = 200**	
Policy	Hit%	TokSave%	Hit%	TokSave%	Hit%	TokSave%
LRU	5.2 ±0.3	3.1 ±0.4	7.4 ±0.4	4.9 ±0.4	9.7 ±0.5	7.0 ±0.4
FIFO	4.5 ±0.2	3.3 ±0.1	6.3 ±0.2	4.2 ±0.4	8.0 ±0.1	5.3 ±0.3
LFU	**7.1** ±0.4	5.0 ±0.7	**9.6** ±0.5	**7.2** ±0.6	**11.6** ±0.8	8.5 ±0.9
W-TinyLFU	6.4 ±0.7	4.2 ±0.9	7.8 ±0.4	5.3 ±0.6	10.0 ±0.2	7.7 ±0.3
W-TinyLFU+Cost	7.2 ±0.3	**6.0** ±0.4	8.6 ±0.4	6.9 ±0.7	9.8 ±0.3	**8.5** ±0.5

LFU achieves the highest raw hit rate on real data. However, **W-TinyLFU + Cost achieves the highest token saving ratio** at cs = 50 (6.0 ±0.4% vs. LRU 3.1 ±0.4%)—a **94% relative improvement** with non-overlapping std ranges.

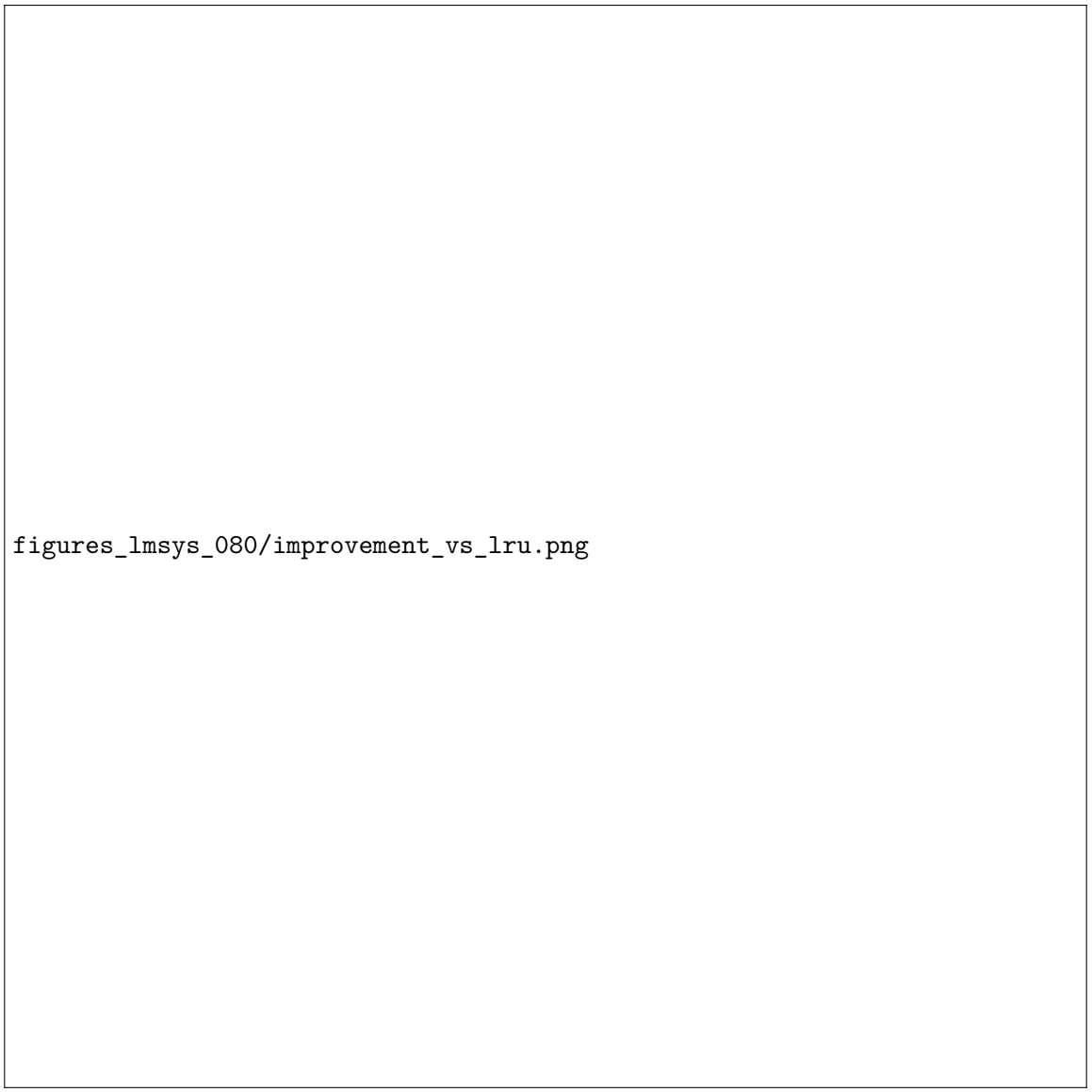
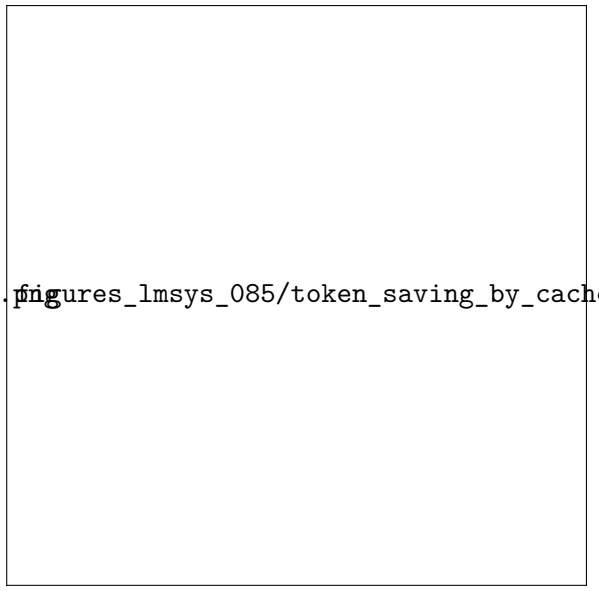


Figure 8: Improvement over LRU on LMSYS data (threshold = 0.80).

4.4 LMSYS-Chat-1M (threshold = 0.85)

This experiment tests robustness to threshold changes.

(a) Hit rate (threshold = 0.85).



(b) Token savings (threshold = 0.85).

Figure 9: LMSYS results at stricter threshold (0.85). Hit rates drop ∼40% vs. 0.80; W-TinyLFU+Cost leads token savings at cs = 50, while LFU leads at larger caches.

Table 7: LMSYS results (mean ± std, 3 trials, threshold = 0.85).

	cs = 50		**cs = 100**		**cs = 200**	
Policy	Hit%	TokSave%	Hit%	TokSave%	Hit%	TokSave%
LRU	3.5 ±0.1	1.7 ±0.1	5.2 ±0.1	2.7 ±0.3	7.1 ±0.1	3.6 ±0.2
FIFO	2.9 ±0.1	1.5 ±0.1	4.2 ±0.1	2.1 ±0.1	6.1 ±0.1	3.1 ±0.3
LFU	**5.4** ±0.5	2.4 ±0.2	**8.3** ±0.4	**5.4** ±0.4	**9.1** ±0.4	**5.4** ±0.3
W-TinyLFU	4.2 ±0.7	2.7 ±1.1	6.0 ±1.3	3.2 ±1.3	7.2 ±0.8	4.1 ±0.5
W-TinyLFU+Cost	4.4 ±1.0	**3.0** ±0.8	5.7 ±0.6	3.6 ±0.4	7.9 ±1.2	4.9 ±1.1

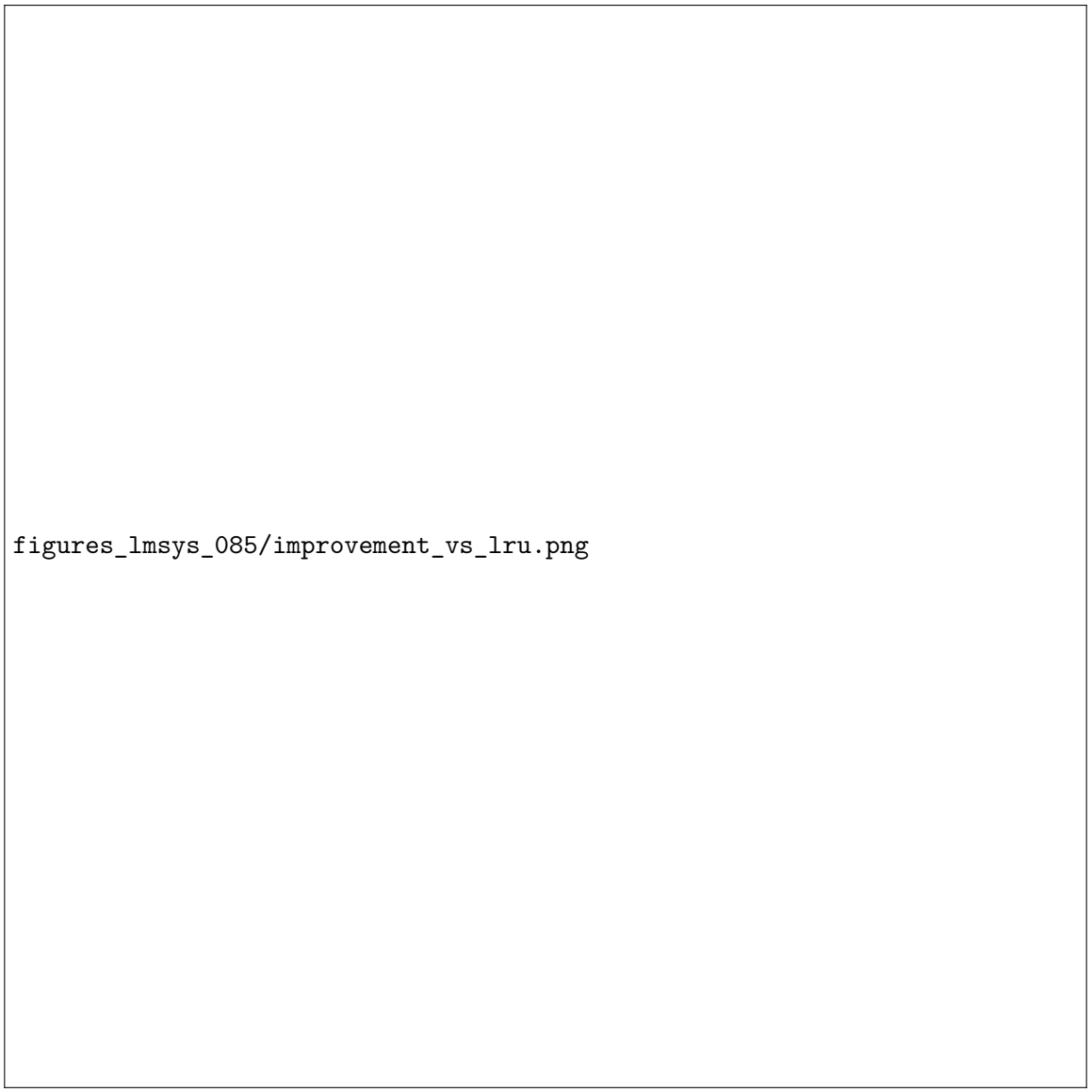


Figure 10: Improvement over LRU on LMSYS data (threshold = 0.85).

Table 8: Threshold sensitivity comparison (cs = 200, mean ± std).

Metric	**t = 0.85**	**t = 0.80**	**Change**
W-TinyLFU+Cost hit rate	7.9 $\pm$1.2%	9.8 $\pm$0.3%	+24%
W-TinyLFU+Cost token save	4.9 $\pm$1.1%	8.5 $\pm$0.5%	+73%
LRU hit rate	7.1 $\pm$0.1%	9.7 $\pm$0.5%	+37%
LRU token save	3.6 $\pm$0.2%	7.0 $\pm$0.4%	+94%
W-TinyLFU+Cost advantage over LRU	**+36%**	**+21%**	**Stable**

The relative advantage is preserved at both thresholds (21–36%), confirming robustness. We recommend threshold 0.80 per SOTA findings [2].

4.5 Ablation Study

4.5.1 Workload Profile Comparison

Table 9: Performance under extreme workload profiles (cache size = 20).

Profile	Policy	Hit Rate	Token Save
Repetitive short	LRU	84.9%	92.9%
(vocab = 100, $\alpha = 1.5$)	W-TinyLFU+Cost	**89.5%**	**97.1%**
Novel long	LRU	5.9%	6.0%
(vocab = 5 000, $\alpha = 0.5$)	W-TinyLFU+Cost	**7.7%**	**9.6%**

Under novel workloads, W-TinyLFU+Cost provides a **+60% relative improvement** in token savings (9.6% vs. 6.0%).

4.5.2 Component Ablation

Table 10: Component ablation isolating cost-awareness (cache size = 20, synthetic).

Policy	Hit Rate	Token Save	vs LRU Hit
LRU (baseline)	15.6%	23.3%	–
FIFO	14.0%	20.5%	−10.2%
LFU	24.2%	39.6%	+55.1%
W-TinyLFU (no cost)	26.0%	41.8%	+66.7%
W-TinyLFU + Cost	**28.7%**	**48.7%**	**+84.0%**

The frequency-based admission filter provides +66.7% over LRU. Adding cost-awareness provides a further **+6.9 pp** in token savings (41.8% → 48.7%).

4.5.3 Window Size Sweep

Table 11: Window size parameter sweep (cache size = 20, synthetic).

Window %	Hit Rate	Token Save	Notes
0.5%	28.5%	48.2%	Slightly below optimal
1.0%	**28.9%**	**49.2%**	**Paper default, optimal**
2.0%	28.9%	49.2%	Equivalent
5.0%	28.9%	49.2%	Robust
10.0%	27.0%	45.7%	Degradation begins
20.0%	27.4%	45.9%	Excess window

The 1% default matches the TinyLFU paper recommendation. Performance is robust from 0.5–5%.

5 Discussion

5.1 Trade-offs

Hit rate vs. latency: W-TinyLFU adds ∼38–43% overhead at p95 for small caches where eviction is frequent, narrowing to 2–6% CPU overhead at cs = 200. In LLM deployments, this overhead (∼40 ms worst case) is negligible compared to API calls (100–2 000 ms). The higher hit rate means fewer API calls, yielding net latency reduction.

Cost-awareness value: Token saving ratio consistently exceeds hit rate in absolute terms. At cs = 50 (synthetic), W-TinyLFU+Cost achieves 53.8% hit rate but 76.4% token savings—the 23 pp gap confirms the policy preferentially retains expensive responses, caching *better*, not just *more*.

Synthetic vs. real data: Synthetic Zipfian data shows dramatic improvements (76.4% vs. 44.0% token savings) because skewed distributions create strong frequency signals. Real LMSYS data shows smaller absolute improvements (6.0% vs. 3.1% at cs = 50) but consistent relative gains (+94%). Lower absolute hit rates on LMSYS reflect query diversity, not policy weakness.

Admission filter vs. workload diversity: The TinyLFU admission filter provides its largest gains under skewed access patterns (synthetic Zipfian), where one-hit-wonder rejection is critical. On real LMSYS data at larger cache sizes (cs ≥ 100), query diversity is high and most queries are unique, leaving fewer one-hit-wonders to filter; here plain LFU becomes competitive on token savings (e.g. 7.2% vs. 6.9% at cs = 100, threshold = 0.80, with overlapping std ranges). This suggests that LLM conversation workloads may benefit from a larger window segment or relaxed admission threshold. In all cases, both W-TinyLFU variants substantially outperform GPTCache's default LRU.

5.2 Sensitivity to Parameters

The policy is robust: window size 0.5–5% performs equivalently; threshold 0.80 and 0.85 both show stable advantage; cache sizes 50–200 all benefit.

5.3 Correctness Verification

Implementation verified against the TinyLFU paper [1] and Caffeine's source code (Table 12). **38 unit tests** verify all components: CMS (7), doorkeeper (6), SLRU (9), W-TinyLFU (10), existing GPTCache (6).

Table 12: Implementation correctness verification against paper and Caffeine source.

Aspect	Paper / Caffeine	Our Implementation	Match
Window / Main split	1% / 99%	1% / 99%	✓
Probation / Protected	20% / 80%	20% / 80%	✓
CMS reset mask	`0x7777...7777`	`0x7777...7777`	✓
Doorkeeper cleared on reset	Yes (paper)	Yes	✓
Admission hash-DoS threshold	6, ∼1/128	6, ∼1/128	✓
Sample size for reset	10× capacity	10× capacity	✓

5.4 Reproducibility

Three paths: (1) Docker: `docker build && docker run`, (2) Local with parallel workers (∼5 min), (3) Conda via `environment.yml`. GitHub Actions CI runs on manual trigger.

6 Conclusion & Future Work

We extended W-TinyLFU with token-cost-aware admission for LLM semantic caching, where response regeneration costs vary by orders of magnitude. The extension multiplies each candidate's TinyLFU frequency estimate by its response token count, biasing eviction decisions toward retaining expensive entries.

Key results vs. GPTCache's default LRU:

- **Synthetic**: 76.4 ±3.5% token savings vs. 44.0 ±0.2% for LRU at cs = 50 (+74% relative)
- **LMSYS (t = 0.80)**: 6.0 ±0.4% vs. 3.1 ±0.4% for LRU at cs = 50 (+94% relative)
- **Ablation**: Cost-awareness adds +6.9 pp token savings on top of frequency-based admission alone
- **Consistency**: Non-overlapping std ranges across 3 independent trials with shuffled query order

An interesting domain-specific observation is that the TinyLFU admission filter provides its largest gains under skewed (Zipfian) access patterns, where one-hit-wonder filtering is critical. On real LMSYS conversations—where query diversity is high and most queries are unique—the admission filter has fewer one-hit-wonders to reject, and plain LFU becomes competitive at larger cache sizes. This suggests that LLM conversation workloads may warrant a larger window segment or a lower admission threshold to better accommodate their high-diversity access patterns.

Future work: Size-aware capacity management, adaptive window sizing via hill climbing, TTL integration, upstream PR to GPTCache.

References

[1] G. Einziger, R. Friedman, and B. Manes, "TinyLFU: A Highly Efficient Cache Admission Policy," *ACM Trans. Storage*, vol. 13, no. 4, art. 35, pp. 35:1–35:31, Dec. 2017. DOI: 10.1145/3149371. arXiv:1512.00727.

[2] S. Regmi and C. P. Pun, "GPT Semantic Cache: Reducing LLM Costs and Latency via Semantic Embedding Caching," arXiv preprint, Nov. 2024. arXiv:2411.05276.

[3] L. Zheng et al., "LMSYS-Chat-1M: A Large-Scale Real-World LLM Conversation Dataset," in *Proc. ICLR*, 2024. arXiv:2309.11998.

[4] B. Manes, "Caffeine: A high performance caching library for Java." github.com/ben-manes/caffeine.

[5] Zilliz, "GPTCache: Semantic cache for LLMs." github.com/zilliztech/GPTCache.

A Repository Structure

```
gptcache/manager/eviction/
    wtinylfu_eviction.py -- W-TinyLFU policy (223 lines)
    count_min_sketch.py -- 4-bit packed CMS (112 lines)
    doorkeeper.py -- Bloom filter doorkeeper (67 lines)
    segmented_lru.py -- Segmented LRU (100 lines)
    manager.py -- Factory registration

benchmarks/
    run_benchmarks.py -- CLI runner (parallel + repeats)
    run_ablation.py -- Ablation study
    simulator.py -- Cache workload simulator
```

```
    data_loader.py -- Dataset loaders
    visualize.py -- Figure generation
    metrics.py -- Metrics collection

tests/unit_tests/eviction/ -- 38 unit tests
.github/workflows/ -- CI pipeline
Dockerfile -- Docker reproducibility
environment.yml -- Conda environment
README_BENCHMARK.md -- Benchmark guide

deliverables/ -- All project deliverables
    README.md -- Checklist mapping guidelines to repo
    report.tex -- This report (source)
    figures_synthetic/ -- Plots (synthetic workload)
    figures_lmsys_080/ -- Plots (LMSYS t=0.80)
    figures_lmsys_085/ -- Plots (LMSYS t=0.85)
    results_synthetic/ -- Raw JSON results + per-request logs
    results_lmsys_080/ -- Raw JSON results + per-request logs
    results_lmsys_085/ -- Raw JSON results + per-request logs
```

B Full Metrics Tables

Table 13: Complete metrics – Synthetic (threshold = 0.85, mean across 3 trials).

Policy	Size	HitRate	TokSave	p50	p95	p99	Mean	QPS	Mem MB
LRU	50	0.306	0.440	39.0	111.9	201.1	47.8	14.9	1.7
FIFO	50	0.276	0.386	39.0	117.0	201.5	48.2	15.0	1.7
LFU	50	0.505	0.717	35.9	96.4	181.7	42.9	14.2	1.6
W-TinyLFU	50	0.504	0.703	51.2	159.6	247.2	63.9	10.8	1.8
W-TinyLFU+Cost	50	0.538	0.764	46.5	154.0	236.8	61.3	11.0	1.8
LRU	100	0.496	0.673	37.4	82.5	167.7	43.2	13.2	1.4
FIFO	100	0.443	0.580	39.0	86.3	173.4	45.1	12.8	1.4
LFU	100	0.595	0.798	36.6	80.7	159.8	42.8	12.6	1.4
W-TinyLFU	100	0.685	0.854	50.6	109.0	172.4	54.7	12.0	1.5
W-TinyLFU+Cost	100	0.713	0.889	44.8	113.9	175.8	52.6	11.5	1.5
LRU	200	0.825	0.931	42.7	74.0	109.0	43.5	12.7	1.3
FIFO	200	0.699	0.809	41.2	71.9	103.1	42.1	14.3	1.4
LFU	200	0.835	0.947	39.3	70.3	96.6	40.2	14.5	1.3
W-TinyLFU	200	0.935	0.983	18.4	52.7	75.2	21.5	20.2	1.3
W-TinyLFU+Cost	200	0.919	0.977	32.6	65.8	89.1	34.1	15.7	1.4

Table 14: Complete metrics – LMSYS-Chat-1M (threshold = 0.80, mean across 3 trials).

Policy	Size	HitRate	TokSave	p50	p95	p99	Mean	QPS	Mem MB
LRU	50	0.052	0.031	37.3	115.0	178.6	47.2	19.7	2.3
FIFO	50	0.045	0.033	37.2	115.4	180.5	47.3	19.7	2.4
LFU	50	0.071	0.050	36.9	114.6	179.8	46.7	19.6	2.3
W-TinyLFU	50	0.064	0.042	74.5	168.5	234.0	86.2	11.0	2.5
W-TinyLFU+Cost	50	0.072	0.060	74.5	169.3	227.8	85.9	11.0	2.5
LRU	100	0.074	0.049	40.2	102.4	174.0	47.0	18.8	2.2
FIFO	100	0.063	0.042	42.5	109.6	185.4	49.8	17.9	2.2
LFU	100	0.096	0.072	42.1	103.9	190.3	49.0	17.6	2.2
W-TinyLFU	100	0.078	0.053	69.6	136.4	192.1	70.9	12.7	2.3
W-TinyLFU+Cost	100	0.086	0.069	74.6	145.7	202.9	79.7	11.4	2.3
LRU	200	0.097	0.070	40.0	60.6	165.6	43.3	17.8	2.1
FIFO	200	0.080	0.053	39.2	63.2	155.5	42.6	18.8	2.1
LFU	200	0.116	0.085	38.6	59.1	154.9	41.7	18.5	2.1
W-TinyLFU	200	0.100	0.077	30.4	64.5	99.4	33.3	23.2	2.4
W-TinyLFU+Cost	200	0.098	0.085	41.5	88.6	133.9	44.3	18.4	2.3

Table 15: Complete metrics – LMSYS-Chat-1M (threshold = 0.85, mean across 3 trials).

Policy	Size	HitRate	TokSave	p50	p95	p99	Mean	QPS	Mem MB
LRU	50	0.035	0.017	44.9	103.9	186.0	50.8	18.9	2.4
FIFO	50	0.029	0.015	45.0	105.7	186.8	50.8	19.0	2.4
LFU	50	0.054	0.024	44.9	107.3	181.7	51.6	18.2	2.4
W-TinyLFU	50	0.042	0.027	71.7	166.7	228.5	82.1	11.6	2.4
W-TinyLFU+Cost	50	0.044	0.030	71.7	166.6	222.5	81.8	11.6	2.4
LRU	100	0.052	0.027	43.9	97.5	175.3	50.1	17.9	2.2
FIFO	100	0.042	0.021	43.3	105.5	191.1	50.4	17.8	2.1
LFU	100	0.083	0.054	42.6	101.9	187.6	49.3	17.5	2.2
W-TinyLFU	100	0.060	0.032	79.0	182.8	262.6	90.4	10.2	2.3
W-TinyLFU+Cost	100	0.057	0.036	80.2	183.4	267.3	93.2	9.9	2.3
LRU	200	0.071	0.036	46.2	91.1	196.6	52.8	15.7	2.1
FIFO	200	0.061	0.031	50.4	92.5	209.8	55.6	15.1	2.1
LFU	200	0.091	0.054	49.2	91.3	213.7	54.4	15.0	2.2
W-TinyLFU	200	0.072	0.041	26.6	75.2	122.2	33.4	22.2	2.3
W-TinyLFU+Cost	200	0.079	0.049	46.0	127.1	202.0	55.7	14.9	2.3

C Sample JSON Output

Each benchmark run produces a JSON result file. Representative example:


```
{
  "policy": "wtinylfu", "cache_size": 50,
  "similarity_threshold": 0.85, "dataset": "synthetic",
  "n_queries": 2900, "n_hits": 1234, "n_misses": 1666,
  "hit_rate": 0.4255, "token_saving_ratio": 0.6986,
  "latency_p50": 44.0, "latency_p95": 91.3,
  "latency_p99": 146.8, "latency_mean": 46.14,
  "throughput_qps": 17.9, "peak_memory_mb": 1.8,
  "cpu_user_seconds": 99.5, "cpu_system_seconds": 63.7,
  "total_evictions": 0, "wall_time_seconds": 162.5
}
```

Per-request logs (`*_log.json`) are also saved alongside each result file, containing individual query metrics used for CDF plots and time-series analysis.

D How to Reproduce

```
git clone https://github.com/a1amit/GPTCache.git
cd GPTCache && git checkout feature/wtinylfu-cost-aware

# Option 1: Docker
docker build -t gptcache-bench . && docker run --rm gptcache-bench

# Option 2: Local (recommended, ~5 min)
python -m venv .venv
# Linux/macOS: source .venv/bin/activate
# Windows: .venv\Scripts\activate
pip install -e . && pip install -r requirements-bench.txt
python -m pytest tests/unit_tests/eviction/ -v -o "addopts=" \
    --ignore=tests/unit_tests/eviction/test_distributed_cache.py
python benchmarks/run_benchmarks.py --dataset synthetic \
    --n_samples 3000 --cache_sizes "50,100,200" \
    --policies "lru,fifo,lfu,wtinylfu,wtinylfu_nocost" \
    --thresholds 0.85 --output results/ --workers -1 --repeats 3
python benchmarks/visualize.py --input results/ --output figures/
```